PHP Control Structurers

Outline

- Introduction
- The if Statement
- The if…..else Statement
- The if….elseif….else Statement
- The Switch Statement
- The While Statement
- The Do….While Statement
- The for Statement
- The break statement

Control Structure

In this lesson we have two different types of structures.

- Conditional Structure
- Iterative Control Structure.

Conditional structure describe the if-then, if-then-else, if-then-else- if-then-else, and switch statements. These structure enable you to make decisions based on single variables or collections of variables.

PHP - The if Statement

- The if statement is used to execute some code **only if a specified condition is true**.
- Syntax:

```
if (condition) {
  code to be executed if condition is true;
}
```

```
<?php
$t = date("H");

if ($t < "20") {
    echo "Have a good day!";
}
?>
```

PHP - The if...else Statement

- Use the if....else statement to execute some code **if a condition is true and another code if the condition is false**.
- Syntax

```
if (condition) {
    code to be executed if condition is true;
} else {
    code to be executed if condition is false;
}
```

```
<?php
$t = date("H");

if ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

PHP - The if...elseif....else Statement

- Use the if....elseif...else statement to **specify a new condition to test, if the first condition is false.**
- Syntax
- if (*condition*) {
  code to be executed if condition is true;
  } elseif (*condition*) {
  code to be executed if condition is true;
  } else {
  code to be executed if condition is false;
  }

```
<?php
$t = date("H");

if ($t < "10") {
    echo "Have a good morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

PHP Switch Statement

- The switch statement accepts one formal parameter, which should be either an integer, float or string primitive variable.
- The break statement is required in each case statement to signal that the evaluation has found a match and should exit the switch statement.
- If there is no break statement in a case, the program will fall through once it has found a match until it runs the default case statements.

The PHP switch Statement

- Use the switch statement to **select one of many blocks of code to be executed**.
- switch (*n*) {
  case *label1:*
  code to be executed if n=label1;
  break;
  case *label2:*
  code to be executed if n=label2;
  break;
  case *label3:*
  code to be executed if n=label3;
  break;
  ...
  default:
  code to be executed if n is different from all labels;
  }

```
<?php
$favcolor = "red";

switch ($favcolor) {
    case "red":
        echo "Your favorite color is red!";
        break;
    case "blue":
        echo "Your favorite color is blue!";
        break;
    case "green":
        echo "Your favorite color is green!";
        break;
    default:
        echo "Your favorite color is neither red, blue, or green!";
}
?>
```

PHP While Statement

- ***While loops*** are the simplest type of loop in PHP. They behave just like their **C**counter parts. The basic form of a *while statement* is:
- **Syntax:-**
  while (expression) {
  // do something }
- The meaning of a while statement is simple. It tells PHP to execute the nested statement(s) repeatedly, as long as the while expression evaluates to **TRUE**. The value of the expression is checked each time at the beginning of the loop, so even if this value changes during the execution of the nested statement(s), execution will not stop until the end of the iteration (each time PHP runs the statements in the loop is one iteration).

```
<?php
$i = 1;
while ($i <= 10) {
echo $i;
$i++;
}
```

Do.. While Statement

- ***Do..while loops*** are very similar to while loops, except the truth expression is checked at the end of each iteration instead of in the beginning. The main difference from regular while loops is that the first iteration of a *do..while loop* is guaranteed to run (the truth expression is only checked at the end of the iteration), whereas it's may not necessarily run with a regular while .
- **Syntax:-**
- do {
  // code to be executed
  } while (expression);

```
<?php
$num = 1;
do {
 echo"Execution number:
$num<br>\n";
$num++;
} while ( $num > 200 && $num <
400 );
?>
```

For Statement

- ***For loops*** are the most complex loops in PHP. They behave like their **C**counterparts. The syntax of a for loop is:.
- **Syntax:-**
- for (initialization expr1; test expr2; modification expr3)
  { // code to be executed }
- The first expression (*expr1*) is evaluated (executed) once unconditionally at the beginning of the loop.
- In the beginning of each iteration, *expr2* is evaluated. If it evaluates to **TRUE**, the loop continues and the nested statement(s) are executed. If it evaluates to**FALSE**, the execution of the loop ends.
- At the end of each iteration, *expr3* is evaluated (executed).
- Each of the expressions can be empty. *expr2* being empty means the loop should be run indefinitely (PHP implicitly considers it as **TRUE**, like **C**). This may not be as useless as you might think, since often you'd want to end the loop using a conditional *break statement* instead of using the for truth expression.

```
<?php
/* example 1 */

for ($i = 1; $i <= 10; $i++) {
    echo $i;
}
/* example 2 */

for ($i = 1; ; $i++) {
    if ($i > 10) {
        break;
    }
    echo $i;
}
```

PHP foreach Loop

- The foreach loop - Loops through a block of code for each element in an array or each property in an object

```
<!DOCTYPE html>
<html>
<body>

<?php
$colors = array("red", "green", "blue",
"yellow");

foreach ($colors as $x) {
  echo "$x <br>";
}
?>

</body>
</html>
```

Break statement

- Break ends execution of the current for, foreach, while, do..while or switch structure.

```
<?php
echo "<p><b>Example of using
the Break statement:</b></p>";

for ($i=0; $i<=10; $i++) {
   if ($i==3)break;
   echo "The number is ".$i;
   echo "<br />";
}
?>
```

Continue statement

- Stops the current iteration and continues to the next.
- Stop, and jump to the next iteration if `$i` is 3:

```
<?php
echo "<p><b>Example of using
the Break statement:</b></p>";

for ($i=0; $i<=10; $i++) {
   if ($i==3)break;
   echo "The number is ".$i;
   echo "<br />";
}
?>
```